perplexity

2PL vs. MVCC

Two-Phase Locking (2PL) controls concurrency by acquiring locks before accessing data. **MVCC** maintains multiple row versions so readers can use a consistent snapshot without waiting for most writers. [1] [2]

2PL lock table

A lock manager typically maintains a table like this:

Data item	Lock mode	Granted to	Waiting transactions
A	S	T1, T2	—
B	X	T3	T4, T5

Where:

- `S` = shared/read lock.
- `X` = exclusive/write lock.
- Multiple transactions can hold `S` on the same item.
- An `X` lock conflicts with both `S` and `X`.
- Waiting transactions are queued until the conflicting lock is released.

Lock compatibility matrix

Requested / Existing	S	X
S	Compatible	No
X	No	No

2PL phases

1. **Growing phase:** the transaction may acquire locks but cannot release any.
2. **Shrinking phase:** after releasing its first lock, it may release locks but cannot acquire new ones.

This guarantees conflict serializability. In **strict 2PL**, write locks are held until commit or rollback, preventing dirty reads and cascading rollbacks. [1]

MVCC behavior

With MVCC, a row may have several versions:

```
A:
  version 1: value = 100, valid until T10
  version 2: value = 150, created by T10
```

A transaction reads the version visible in its snapshot:

```
T1 starts
T2 updates A from 100 to 150
T1 continues reading A → still sees 100
T2 commits
```

Therefore, under MVCC:

- Reads generally do not require shared locks.
- Readers and writers usually do not block each other.
- Updates create new versions rather than overwriting the version currently read.
- Writers can still need exclusive locks or conflict detection.
- Old versions must eventually be cleaned up.

MVCC improves read concurrency, but it does not automatically prevent every anomaly. For example, snapshot isolation can allow **write skew**; stronger serializable validation or predicate/range locking may be required. [3] [4]

Practical comparison

Aspect	2PL	MVCC
Main strategy	Lock data before access	Read appropriate data version
Read/write blocking	Common	Usually avoided
Deadlocks	Possible	Reduced, but write conflicts remain
Storage	One current version plus lock metadata	Multiple row versions
Dirty reads	Prevented by appropriate isolation	Normally prevented
Serializable isolation	Natural with strict 2PL	Requires additional validation or locking
Main cost	Waiting and deadlocks	Version storage and garbage collection

Rule of thumb: use 2PL when explicit locking and strong serializability are central; use MVCC when the workload has many concurrent reads and you want readers to proceed while writers are active. Many database systems combine both approaches rather than using them exclusively. [2] [5]

⁂

1. https://www.databricks.com/blog/concurrency-control
2. https://stormatics.tech/blogs/database-concurrency-two-phase-locking-2pl-to-mvcc-part-1
3. https://postgrespro.com/blog/pgsql/5967856
4. https://vladmihalcea.com/write-skew-2pl-mvcc/
5. https://hsqldb.org/doc/guide/sessions-chapt.html
6. https://dzone.com/articles/all-about-two-phase-locking
7. https://hudi.apache.org/blog/2025/01/28/concurrency-control/
8. https://github.com/manhdaovan/til/issues/21
9. https://www.phoenixdata.ai/glossary/multiversion-concurrency-control
10. https://blog.stackademic.com/understanding-concurrency-control-2pl-vs-mvcc-in-database-systems-2e8af23f1668